

B.M.S COLLEGE OF ENGINEERING BENGALURU
Autonomous Institute, Affiliated to VTU



PLC AAT Report on

WEATHER APP

Submitted in partial fulfillment of the requirements of AAT

Bachelor of Engineering in Computer Science and
Engineering

Submitted by:

VIDHAN TEWATIA
TUSHAR GARG
VISHESH AGGARWAL
VARUN

Department of Computer Science and Engineering
B.M.S College of Engineering
Bull Temple Road, Basavangudi, Bangalore 560019
2025-2026

BMS COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We, Vidhan Tewatia, Tushar Garg, Vishesh Aggarwal, and Varun, students of the 2nd Semester, B.E., Department of Computer Science and Engineering, BMS College of Engineering, Bangalore, hereby declare that this AAT Project entitled “**WEATHER APP**” has been carried out in the Department of CSE, BMS College of Engineering, Bangalore during the academic semester February 2026 – July 2026. We also declare that to the best of our knowledge and belief, the AAT Project report is not from part of any other report by any other students.

Student Name

Student Signature

1. ***Vidhan Tewatia***

2. ***Tushar Garg***

3. ***Vishesh Aggarwal***

4. ***Varun***

BMS COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that AAT project titled ‘**WEATHER APP**’ has been carried out by :
VIDHAN TEWATIA (1BM25CS313)
TUSHAR GARG (1BM25CS305)
VARUN (1WM25CS294)
VISHESH AGGARWAL (1BM25CS314)
during academic year 2025-2026

Faculty Signature

Table of Contents

Sl. No.	Title	Page no.
1	Introduction	5
2	Hardware and Software Requirement	6
3	Design	7
4	Source code	10
5	Results (screenshots)	11
6	Refrences	12

1. INTRODUCTION

Weather plays a crucial role in our daily lives, influencing decisions ranging from travel plans to agricultural activities. With the increasing reliance on technology, mobile and web applications have become essential tools for accessing real-time weather information. This project focuses on the development of a Python-based weather application designed to provide users with accurate and timely weather updates. By integrating APIs and leveraging Python's versatility, the app delivers key details such as temperature, humidity, wind speed, and forecasts in a user-friendly interface. The primary objective of this project was not only to enhance technical skills in Python programming but also to demonstrate the practical application of software development in solving everyday problems. Through this project, I explored concepts such as API handling, data visualization, and error management, ultimately creating a functional and accessible weather app.

2. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 or equivalent
- RAM: 4 GB minimum
- Storage: 100 MB free disk space
- Internet Connection: Required to fetch real-time weather data
- Keyboard and Monitor

Software Requirements:

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Python 3.x
- Python Library: Requests
- API Service: WeatherAPI
- Development Environment: Visual Studio Code / PyCharm / IDLE
- Internet Browser (for API documentation and key generation)

3. DESIGN

The Weather App is designed using a simple and modular approach in Python to fetch and display real-time weather information for a user-provided location. The application follows an input–process–output design model where the user enters a location, the system processes the request using an external weather API, and the weather details are displayed.

3.1 System Design

The application consists of the following components:

1. User Input Module

The user is prompted to enter the name of a city or location. The input is validated to ensure that it is not empty before sending the request.

2. API Request Module

The program uses the Python requests library to communicate with the WeatherAPI service. A request is sent to the API endpoint using an API key and the entered location as parameters.

3. Data Processing Module

After receiving the API response, the application checks the HTTP status code and extracts required information from the JSON response such as:

- Location name
- Country name
- Temperature (°C)
- Weather condition
- Humidity

4. Output Module

The extracted weather information is formatted and displayed clearly to the user in the console. If the location entered is invalid, an error message is displayed.

3.2 Working Principle

1. User enters a location name.
2. Program validates the input.
3. API request is sent to WeatherAPI.
4. Response data is received in JSON format.
5. Weather information is extracted and displayed.
6. Program repeats until manually terminated.

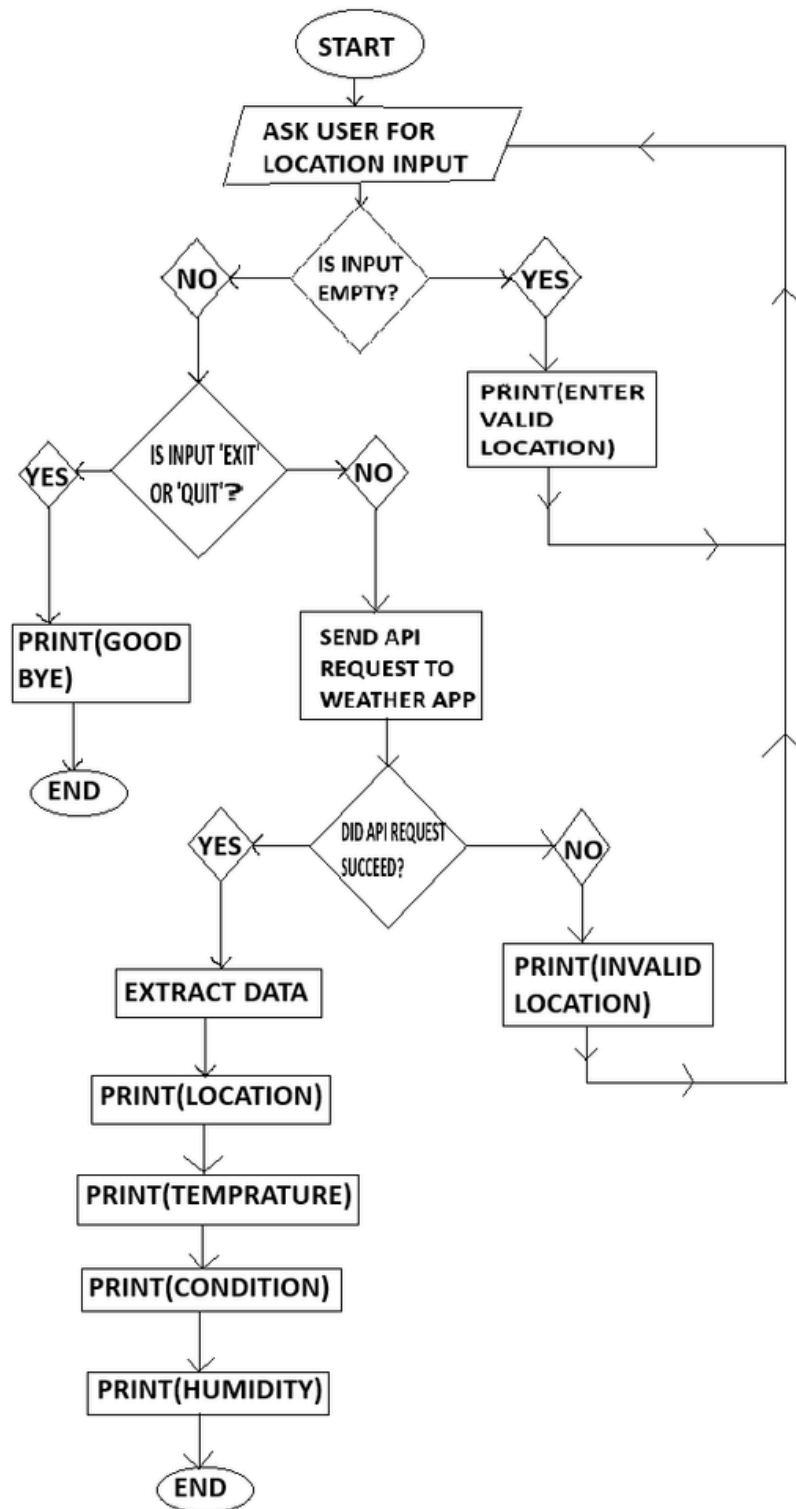
3.3 Design Architecture

Input → Validation → API Request → Data Processing → Weather Information Display

3(a). ALGORITHM

1. Start
2. Initialize Constants:
 - a. 1. Set **`API\_KEY`**.
 - b. 2. Set **`BASE\_URL`** to the Weather API endpoint.
3. Begin Infinite Loop:
 - a. 1. Repeat until the program is manually stopped.
4. Take User Input:
 - a. 1. Prompt the user: *“Enter a location.”*
 - b. 2. Store the input in **`location`**.
5. Validate Input:
 - a. 1. Check if **`location`** is empty.
 - b. 1.1. If empty, display *“Please enter a valid location.”*
 - c. 1.2. Go back to Step 4.
6. Prepare API Request:
 - a. 1. Create a **`params`** dictionary:
 - b. 1.1. **`key`** = **`API\_KEY`**.
 - c. 1.2. **`q`** = **`location`**.
 - d. 2. Send request using **`requests.get(BASE\_URL, params=params)`**.
 - e. 3. Store the response.
7. Check Response Status:
 - a. 1. If **`status\_code == 200`**:
 - b. 1.1. Parse JSON data.
 - c. 1.2. Extract:
 - d. 1.2.1. Location name and country.
 - e. 1.2.2. Temperature (°C).
 - f. 1.2.3. Weather condition.
 - g. 1.2.4. Humidity.
 - h. 1.3. Display formatted weather details.
 - i. 2. Else:
 - j. 2.1. Display *“Please enter a valid location.”*
8. Repeat Loop:
 - a. 1. Return to Step 4.
9. End (Only when the program is terminated manually).

3(b). FLOWCHART



4. SOURCE CODE

```
import requests

API_KEY = <API_KEY>
BASE_URL = "https://api.weatherapi.com/v1/current.json"

while True:
    location = input("Enter a location: ").strip()

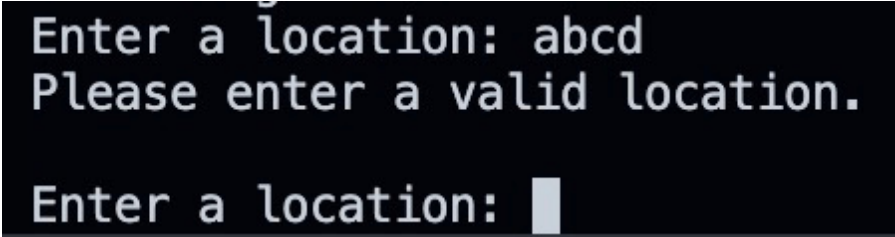
    if not location:
        print("Please enter a valid location.")
        continue

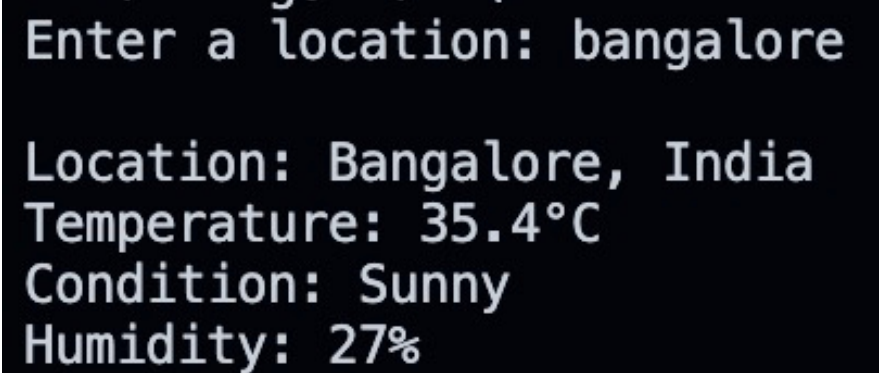
    params = {"key": API_KEY, "q": location}
    response = requests.get(BASE_URL, params=params)

    if response.status_code == 200:
        data = response.json()
        loc = data["location"]
        current = data["current"]

        print(f"\nLocation: {loc['name']}, {loc['country']}")
        print(f"Temperature: {current['temp_c']}°C")
        print(f"Condition: {current['condition']['text']}")
        print(f"Humidity: {current['humidity']}%\n")
    else:
        print("Please enter a valid location.\n")
```

5. RESULTS (SCREESHOTS)

1) A terminal window with a black background and white text. The text reads: "Enter a location: abcd", "Please enter a valid location.", and "Enter a location: " followed by a white cursor bar.

2) A terminal window with a black background and white text. The text reads: "Enter a location: bangalore", "Location: Bangalore, India", "Temperature: 35.4°C", "Condition: Sunny", and "Humidity: 27%".

6. REFERENCES

- [1] WeatherAPI, "Weather API Documentation," [Online]. Available: <https://www.weatherapi.com/docs/>
- [2] Python Software Foundation, "Python Documentation," [Online]. Available: <https://docs.python.org/3/>
- [3] K. Reitz, "Requests: HTTP for Humans," [Online]. Available: <https://docs.python-requests.org/>
- [4] Mozilla, "HTTP Status Codes," [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
- [5] REST API Tutorial, "RESTful Web Services," [Online]. Available: <https://restfulapi.net/>